

**Auckland University of Technology**  
**Data Process and Software Modelling -**  
**ENSE706**

**Assignment 2: Final Project Report**  
*Microtransaction Analytics System (MAS)*  
Organisation: Grinding Gear Games

**Name: Suemon Kwok**  
**Student ID: 14883335**

GitHub: <https://github.com/Suemon-Kwok/ENSE706-Data-Process-and-Software-Modelling-2026-S1---Microtransaction-Analytics-System---14883335>

# Table of Contents

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>Introduction .....</b>                                                     | <b>3</b>  |
| <b>Task 5: Refined Static and Dynamic Modelling .....</b>                     | <b>4</b>  |
| <b>5.1 Refined UML Class Diagram .....</b>                                    | <b>4</b>  |
| <b>5.2 Sequence Diagrams .....</b>                                            | <b>5</b>  |
| <b>5.2.1 Sequence Diagram 1: Export Report (Strategy Pattern, UC12) .....</b> | <b>5</b>  |
| <b>5.2.2 Sequence Diagram 2: Authenticate User (Singleton, UC13).....</b>     | <b>6</b>  |
| <b>5.2.3 Sequence Diagram 3: Create MTX Items (Factory Method) .....</b>      | <b>7</b>  |
| <b>5.3 Activity Diagram: Generate Filterable Sales Report (UC09).....</b>     | <b>8</b>  |
| <b>5.4 State Machine Diagram: Transaction Lifecycle .....</b>                 | <b>9</b>  |
| <b>Task 6: Design Pattern Application and Implementation .....</b>            | <b>10</b> |
| <b>6.1 Pattern 1: Strategy (Behavioural).....</b>                             | <b>10</b> |
| <b>6.2 Pattern 2: Factory Method (Creational) .....</b>                       | <b>11</b> |
| <b>6.3 Pattern 3: Singleton (Creational) .....</b>                            | <b>12</b> |
| <b>6.4 Pattern 4: Adapter (Structural) .....</b>                              | <b>13</b> |
| <b>6.5 Pattern 5: Decorator (Structural) .....</b>                            | <b>14</b> |
| <b>Task 7: Comparative Design Analysis (Before vs After Patterns) .....</b>   | <b>15</b> |
| <b>7.1 Changes to Class Structure and Responsibility Allocation.....</b>      | <b>15</b> |
| <b>7.2 Improvements in Cohesion and Reductions in Coupling.....</b>           | <b>15</b> |
| <b>7.3 Impact on Extensibility and Maintainability .....</b>                  | <b>16</b> |
| <b>7.4 Consequences for C# Implementation Complexity .....</b>                | <b>16</b> |
| <b>Task 8: Critical Reflection .....</b>                                      | <b>17</b> |
| <b>8.1 Design Trade-offs Introduced by Design Patterns .....</b>              | <b>17</b> |
| <b>8.1.1 Strategy: Indirection vs Transparency .....</b>                      | <b>17</b> |
| <b>8.1.2 Singleton: Testability vs Controlled Access .....</b>                | <b>17</b> |
| <b>8.1.3 Adapter: Transitional Risk.....</b>                                  | <b>17</b> |
| <b>8.1.4 Decorator: Stacking Order Dependency .....</b>                       | <b>17</b> |
| <b>8.2 Limitations of the Final Design and Implementation.....</b>            | <b>17</b> |
| <b>8.3 Lessons Learned from Aligning UML Models with C# Code .....</b>        | <b>18</b> |
| <b>Conclusion .....</b>                                                       | <b>19</b> |
| <b>References .....</b>                                                       | <b>20</b> |

# Introduction

Grinding Gear Games (GGG), a New Zealand-based independent developer behind Path of Exile, generates its entire revenue through voluntary microtransaction purchases of cosmetic items. Phase 1 of the Microtransaction Analytics System (MAS) established the foundational architecture for tracking, reporting, and exporting this transaction data. A functional C# implementation was delivered with 18 use cases, a 27-class UML model, and core features including role-based access control, filterable sales reports, revenue trend visualisation, and multi-format export.

Despite this foundation, Phase 1 contained several structural weaknesses that limited the system's long-term maintainability. The export service embedded all format-specific logic in a single switch block, violating the Open/Closed Principle. The authentication service bundled session management with demo data seeding, breaching the Single Responsibility Principle. Item construction was scattered across 20 direct instantiation calls in the seeding layer, creating tight coupling to every concrete subclass. These weaknesses were explicitly identified in Phase 1, Section 4.5, and form the direct motivation for the refactoring undertaken in Phase 2.

This report documents the Phase 2 work across four tasks. Task 5 presents the refined UML models which are an updated class diagram, three sequence diagrams, one activity diagram, and one state machine diagram and each improved to incorporate interfaces, abstraction layers, and GoF pattern structures. Task 6 applies five GoF design patterns to the codebase with supporting pattern structure diagrams. Task 7 critically compares the design before and after pattern application. Task 8 reflects on the trade-offs, limitations, and lessons learned.

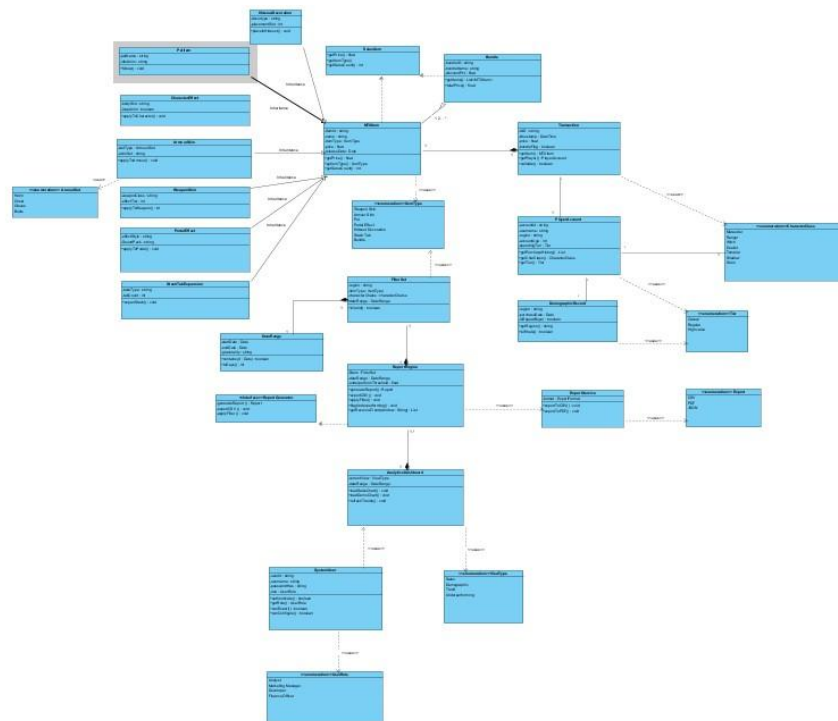


Figure 1. Phase 1 UML class diagram, used as the baseline for Phase 2 refinement.

## Task 5: Refined Static and Dynamic Modelling

Phase 2 begins with a comprehensive refinement of the UML models produced in Phase 1. The Phase 1 class diagram revealed four structural gaps: the absence of a formal export interface, no factory mechanism for item creation, a monolithic authentication class, and no audit logging pathway. Each gap is addressed in the refined class diagram by introducing the appropriate interface or pattern class.

### 5.1 Refined UML Class Diagram

The Phase 1 class diagram contained 27 classes across four layers: item hierarchy, transaction domain, player model, and service layer. The Phase 2 refined diagram preserves this structure and adds six elements driven by GoF pattern application: `IExportStrategy` interface with three concrete strategy classes, `MTXItemFactory` as a dedicated creator, `AuthService` marked as a Singleton, `LoggingReportEngineDecorator` wrapping `IReportGenerator`, and `LegacyExportServiceAdapter` bridging the Phase 1 export API to the new interface.

The most significant structural change is the extraction of `IExportStrategy` from `ExportService`, which in Phase 1 violated the Open/Closed Principle because any new export format required editing an existing class. The refined diagram encodes this split formally: all three concrete strategies realise `IExportStrategy` via dashed realisation arrows, while `ExportService` becomes a pure context class connected to the interface by a dependency arrow. Four previously floating value objects are also now connected. `FilterSet` and `ThresholdConfig` link to `ReportEngine` via dependency arrows, reflecting that `ApplyFilter()` consumes a `FilterSet` and `FlagUnderperformingItems()` reads `ThresholdConfig`. `Report` connects to `ReportEngine` as a produced output, and `DemographicRecord` connects to `Transaction` via aggregation, confirming that each transaction holds a demographic snapshot at purchase time.

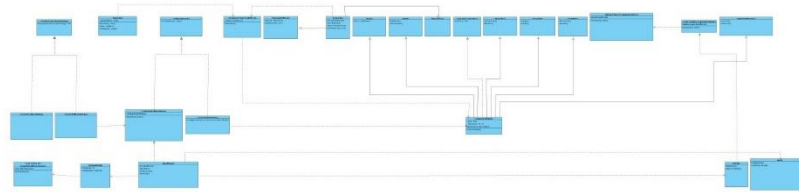


Figure 2. Refined UML class diagram showing `IExportStrategy`, `MTXItemFactory`, Singleton `AuthService`, and `Decorator` added to the Phase 1 baseline.

## 5.2 Sequence Diagrams

### 5.2.1 Sequence Diagram 1: Export Report (Strategy Pattern, UC12)

The first sequence diagram models the export report use case and makes the Strategy pattern's runtime delegation visible. The user selects a format in MainDashboard, which calls `ExportService.SetStrategy(new CsvExportStrategy)`. `ExportService` then calls `Export(report, path)` on the injected `IExportStrategy` reference. `CsvExportStrategy` serialises the Report DTO (Data Transfer Object) and delegates `writeFile` to `FileSystem`. An alt frame separates the permission-denied branch, where no file is written and an access-denied message is returned. `ExportService` never references CSV logic directly; it communicates only through the interface.

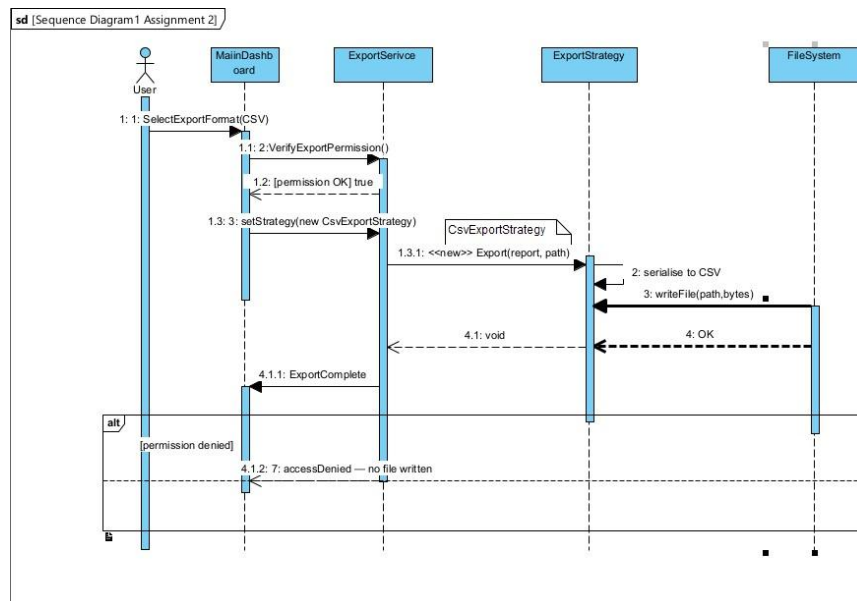


Figure 3. Sequence Diagram 1: Export Report via Strategy pattern, showing runtime delegation to `CsvExportStrategy` (UC12).

## 5.2.2 Sequence Diagram 2: Authenticate User (Singleton, UC13)

The second sequence diagram models the authentication flow and makes the Singleton pattern's enforcement visible. LoginForm calls AuthService.GetInstance() rather than instantiating AuthService directly; the annotation on the return arrow confirms that the same instance is returned on every call. AuthService delegates user lookup to UserRepository, then computes SHA-256(password) as a self-message before branching on hash match or mismatch inside an alt frame. On success, MainDashboardForm is constructed with the authenticated user's role; load RBAC matrix and render role views appear as self-messages on the MainDashboard lifeline, confirming that role-appropriate views are populated before the user interacts with the dashboard.

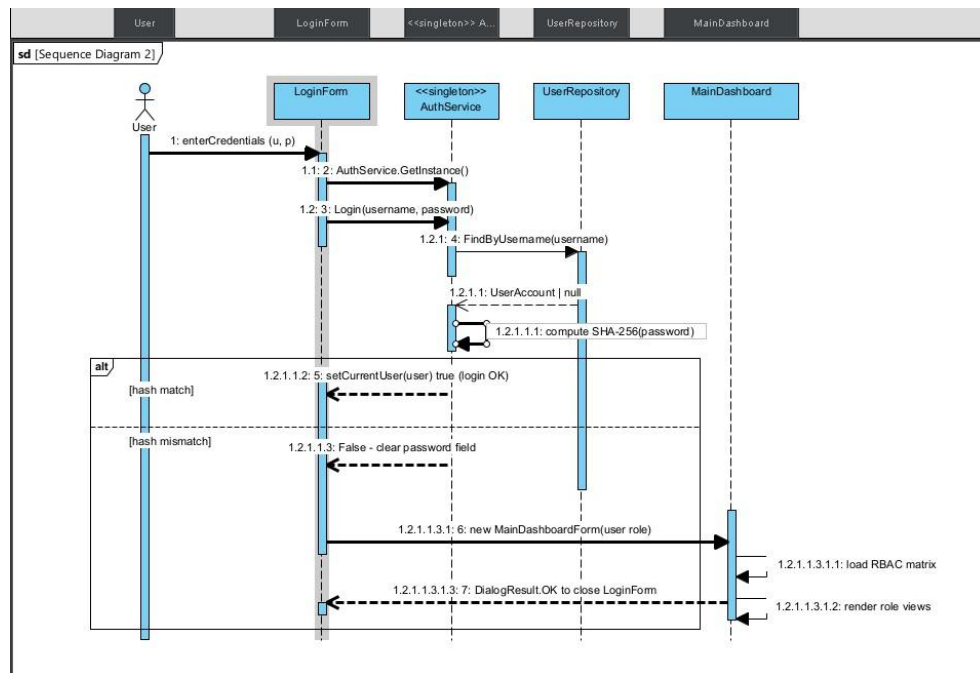


Figure 4. Sequence Diagram 2: Authenticate User via Singleton AuthService, showing GetInstance(), SHA-256 verification, and RBAC loading (UC13).

### 5.2.3 Sequence Diagram 3: Create MTX Items (Factory Method)

The third sequence diagram models the application startup seeding process and demonstrates the Factory Method pattern. Inside a loop frame labelled for each item in seed catalogue, Program calls `MTXItemFactory.Create(ItemType, id, name, price)` rather than constructing concrete types directly. The factory dispatches to the appropriate constructor which the new `WeaponSkin()` or new `PetItem()` which then returns an `MTXItem` reference. Program passes this reference to `ItemRepository.Add(item)`, receiving the confirmation item persisted. The note attached to `MTXItemFactory` states: Factory encapsulates `new()` logic. Program depends only on `ISalesItem`, confirming the decoupling achieved.

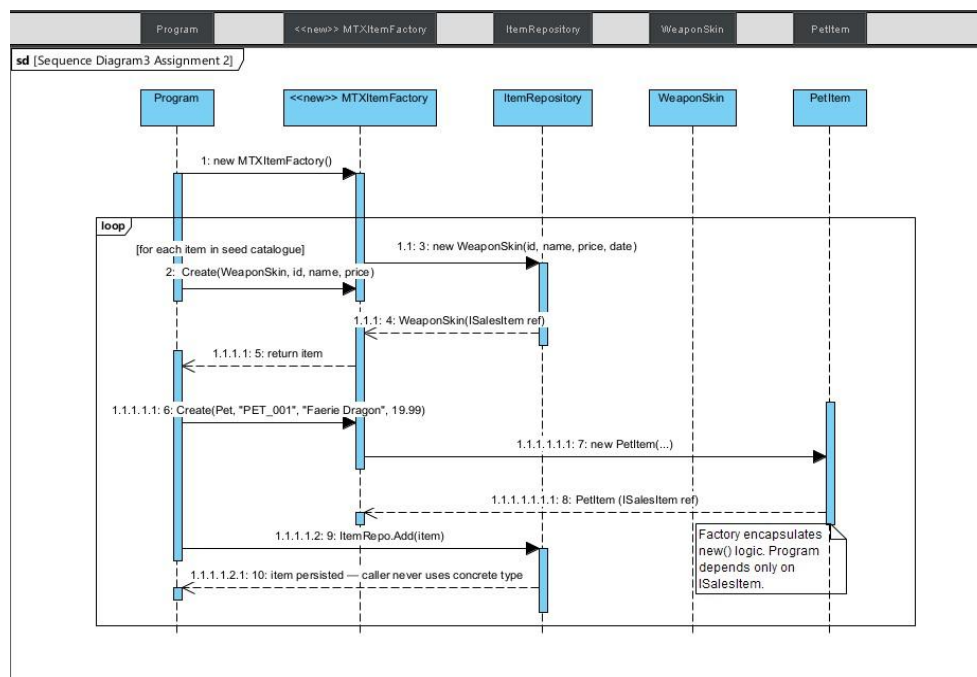


Figure 5. Sequence Diagram 3: MTX item creation via Factory Method, with loop frame and `ISalesItem` return typing (seeding flow).

### 5.3 Activity Diagram: Generate Filterable Sales Report (UC09)

The activity diagram for UC09 uses two swim lanes which are User and System (MAS) these separate actor responsibilities from system processing. The flow begins with the user navigating to the dashboard, triggering session validation. A Session valid? decision diamond branches to Redirect to login (looping back to the start node) or continues to Select filter criteria. A second decision, date range valid?, loops back to the ValidateFilterSet activity on invalid input or proceeds to ApplyFilter + GenerateReport on valid input. The system then flags underperforming items and renders the report for the user to review. A final export? decision branches to ExportService.Export() before both paths converge at the final node.

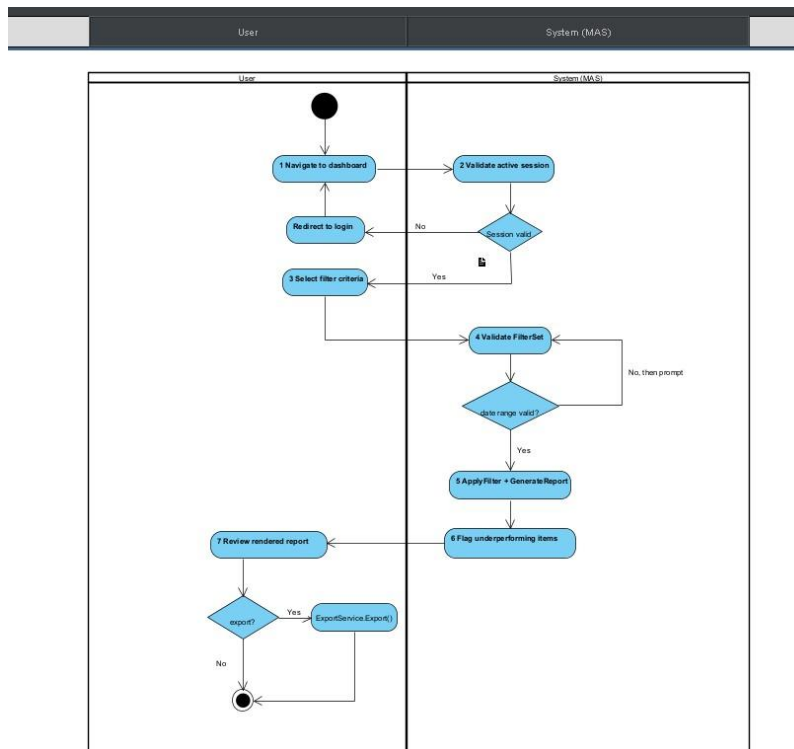


Figure 6. Activity Diagram: Generate Filterable Sales Report (UC09), showing session validation, filter validation, and optional export branch.

## 5.4 State Machine Diagram: Transaction Lifecycle

The transaction state machine models the full lifecycle of a single microtransaction from receipt to archival. The initial pseudo-state transitions immediately to Pending (do/ buffer event, await game server). A purchase event triggers entry to Validating (do/ check itemID, check playerAccount). A Valid? choice pseudo-state branches to Recorded (entry/ createGuid(), do/ increment SalesCount, update SpendingTier) on success, or to Rejected (entry/ logError(), do/ refundPlayer) on failure. From Recorded, a repo OK? decision branches to Persisted on success or to Queued (do/ hold in retry queue, raise NFR02 alert) on repository unavailability, with a retry transition back to Recorded when the repository recovers. From Persisted, an [export requested] guard transitions to Exported (entry/ ExportStrategy.Export(), do/ write file). All terminal states merge at a fork bar before the final pseudo-state.

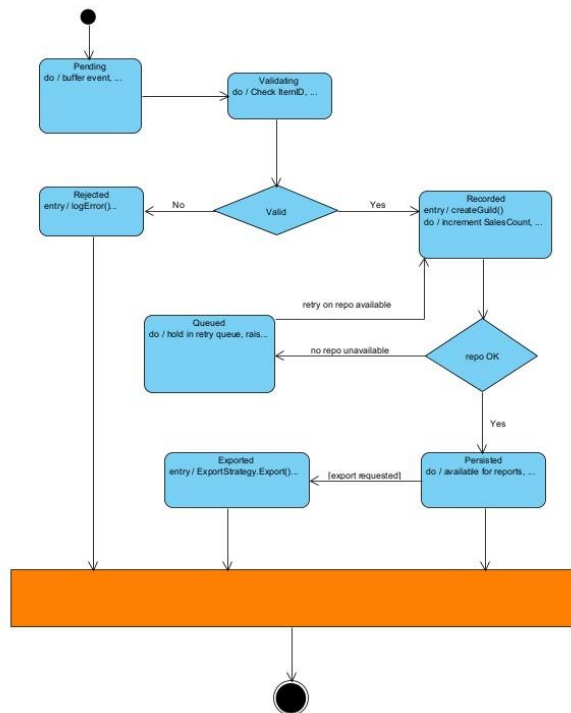


Figure 7. State Machine Diagram: Transaction lifecycle from Pending through Recorded, Queued, Persisted, and Exported.

## Task 6: Design Pattern Application and Implementation

Five GoF design patterns were applied to the MAS in Phase 2. The selection was grounded in the design weaknesses identified in Phase 1, Section 4.5, as documented in the Design Pattern Selection Rationale. Each pattern is described below with its GoF classification, the specific Phase 1 weakness it addresses, its UML structure, and its C# implementation. The lecture material produced by Dr Abubakar Siddique for ENSE706 is referenced throughout to connect each pattern to its formal teaching context.

### 6.1 Pattern 1: Strategy (Behavioural)

The Strategy pattern, classified as a Behavioural GoF pattern, is defined in the course slides as: *"a pattern that lets you define a family of algorithms, place each algorithm in a separate class, and make them interchangeable"* (Siddique, 2026, Slide 12, p.5). It addresses systems that contain multiple algorithms for the same task implemented using large conditional statements that are difficult to maintain.

**Phase 1 weakness:** ExportService.cs (418 lines) embedded CSV, PDF, and JSON serialisation as private static methods inside a switch block. Adding a new format required editing this file, violating the Open/Closed Principle and creating merge-conflict risk for all developers working on exports simultaneously.

**Why Strategy and not Template Method:** Template Method was considered but rejected because the three export formats share no meaningful algorithmic skeleton. CSV, PDF, and JSON serialisation are entirely independent transformations with no shared steps. Strategy is the correct fit because the behaviour must be selectable at runtime based on user choice.

**C# implementation:** IExportStrategy defines Export(Report, string). CsvExportStrategy, PdfExportStrategy, and JsonExportStrategy each implement this interface. ExportService holds \_strategy: IExportStrategy and delegates Export() in one line: \_strategy.Export(report, outputPath). SetStrategy(ExportFormat) resolves the concrete class from an enum, preserving backward compatibility with existing call sites.

**Advantages (per lecture slides):** eliminates complex conditional branching, enables runtime selection of algorithms, improves modularity by isolating behaviour into independent units, and aligns with the Single Responsibility and Open/Closed SOLID principles (Siddique, 2026, Slide 12, p.37).

**Disadvantage:** increases the number of classes and introduces an indirection layer that can make debugging slightly harder for developers unfamiliar with the pattern (Siddique, 2026, Slide 12, p.38).

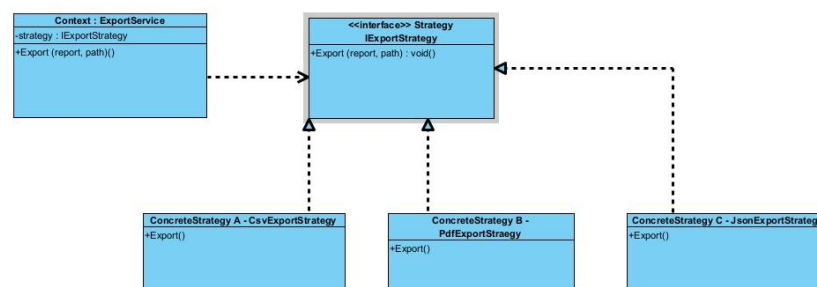


Figure 8. Pattern 1: Strategy pattern structure diagram showing ExportService (Context), IExportStrategy (Strategy interface), and three concrete strategy classes.

## 6.2 Pattern 2: Factory Method (Creational)

The Factory Method pattern is classified as a Creational GoF pattern. The course slides define it as: *"a pattern that defines an interface for creating objects in a superclass, but allows subclasses to decide which specific object to instantiate"* (Siddique, 2026, Slide 09, p.51). It replaces direct new() calls with a dedicated factory method, decoupling client code from concrete implementations.

**Phase 1 weakness:** SeedData.CreateCatalogue() used 20 direct new() calls across seven concrete item types. Program.cs was coupled to every subclass such as WeaponSkin, PetItem, ArmourSkin, PortalEffect, HideoutDecoration, StashTabExpansion, and Bundle. Any constructor signature change broke all 20 call sites simultaneously, and there was no central place to add shared post-construction logic.

**C# implementation:** MTXItemFactory.Create(ItemType, id, name, price, date) resolves the correct subclass from the ItemType enum using a switch expression and returns it typed as MTXItem. Callers receive a base class reference and never reference concrete subclasses. Overloaded variants such as CreateWeaponSkin(), CreateArmourSkin(), CreateBundle() exposes named creation methods for cases where subclass-specific parameters are needed.

**Advantages:** avoids tight coupling between client code and concrete products, provides a single point for post-construction logic, and makes the codebase extensible by adding MountSkin requires one new subclass and one new case in the factory, with zero changes to any caller.

**Disadvantage:** reduces IDE discoverability; a developer searching for where PetItem is created must know to look at the factory rather than searching for new PetItem().

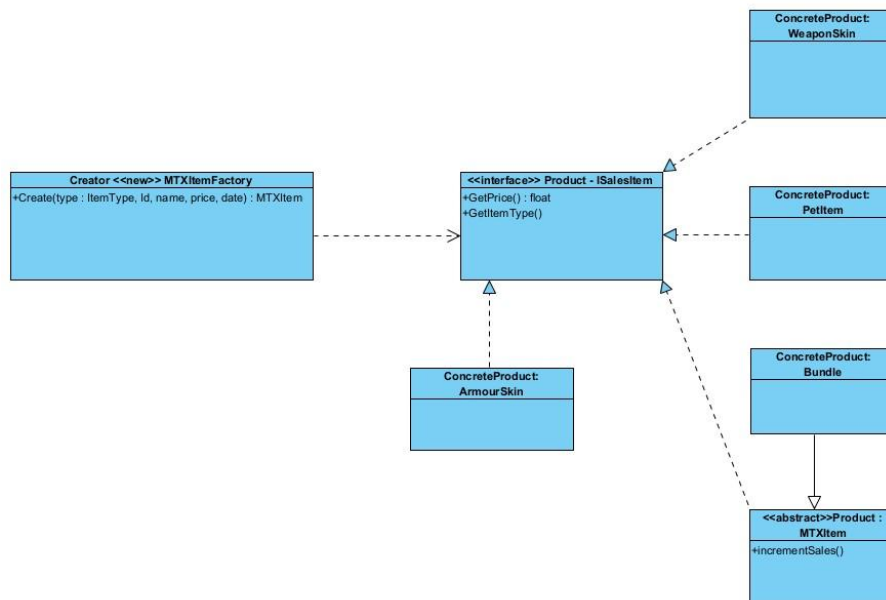


Figure 9. Pattern 2: Factory Method pattern structure diagram showing MTXItemFactory (Creator), ISalesItem and MTXItem (Products), and concrete product classes.

### 6.3 Pattern 3: Singleton (Creational)

The Singleton pattern is classified as a Creational GoF pattern. The course slides define its intent as: *"to ensure that a class has only one instance while providing a global access point to that instance"* (Siddique, 2026, Slide 10, p.2). The slides specifically identify database access objects, logging services, and configuration managers as the canonical use cases where Singleton applies (Siddique, 2026, Slide 10, p.8).

**Why Singleton applies to AuthService:** AuthService functions as the application-wide session store which it holds CurrentUser, verifies credentials, and controls export and configuration permissions. In Phase 1, AuthService was instantiated by Program.cs and passed as a constructor argument to LoginForm and MainDashboardForm. Nothing prevented a second AuthService from being instantiated by a future background service or test harness, which would give it its own CurrentUser where a split-brain session bug directly analogous to the multiple database connection problem described in the lecture (*analogous to the duplicate alert manager problem described in Siddique, 2026, Slide 10, p.11–12*).

**C# implementation:** AuthService is declared sealed with a private constructor. A private static volatile field stores the single instance. GetInstance() uses Lazy<T> for thread safety, as covered in the course slides (Siddique, 2026, Slide 10, p.15–18) this ensures that concurrent calls during application startup do not create duplicate instances. LoginForm and MainDashboard both call AuthService.GetInstance(), which the PatternUsageDemo.cs file proves returns ReferenceEquals() true.

**Advantages (per lecture slides):** ensures a single instance, saves resources by avoiding unnecessary object creation, and provides a centralized global access point without the replacement risk of ordinary global variables (Siddique, 2026, Slide 10, p.5–6, p.9).

**Disadvantage:** introduces global state that makes unit testing difficult, as the static instance persists across tests unless explicitly reset. For a future ASP.NET migration supporting NFR04 (50 concurrent users), the Singleton would need to be replaced with a scoped dependency injection registration per HTTP request.

## 6.4 Pattern 4: Adapter (Structural)

The Adapter pattern is classified as a Structural GoF pattern. The course slides define it as: *"a pattern that allows objects with incompatible interfaces to collaborate" by acting as a translator between two existing systems (Siddique, 2026, Slide 11, p.5)*. The lecture uses a New Zealand power socket adaptor as the real-world analogy: the plug shape changes but neither the charger nor the socket is modified (Siddique, 2026, Slide 11, p.7).

**Phase 1 weakness addressed:** The Phase 1 ExportService required the format to be fixed at construction time: `new ExportService(ExportFormat.CSV)`. The Phase 2 IExportStrategy interface is format agnostic, where the strategy IS the format. These two contracts are incompatible; the old class cannot be used as an IExportStrategy without modification. LegacyExportServiceAdapter wraps LegacyExportService (the Adaptee) and exposes `Export(Report, string)` as required by IExportStrategy (the Target), with delegation occurring transparently inside the adapter.

**Advantages (per lecture slides):** promotes code reuse, avoids modification of the existing system, supports the Open/Closed and Single Responsibility principles, and encapsulates compatibility logic so the client never depends on the Adaptee directly (Siddique, 2026, Slide 11, p.29-30).

**Disadvantage:** adds an extra abstraction layer and can hide poor architectural decisions by making legacy code appear compatible when it should be rewritten (Siddique, 2026, Slide 11, p.31). The adapter is intended as a transitional measure.

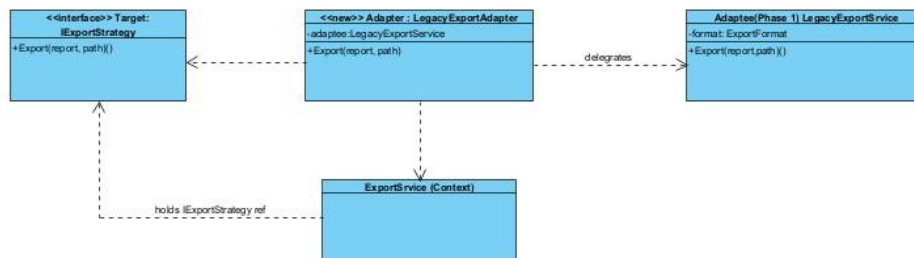


Figure 10. Pattern 4: Adapter pattern structure diagram showing LegacyExportServiceAdapter bridging IExportStrategy (Target) and LegacyExportService (Adaptee).

## 6.5 Pattern 5: Decorator (Structural)

The Decorator pattern is classified as a Structural GoF pattern. The course slides define it as a pattern that *"allows behaviour to be added to individual objects dynamically without modifying their class"* (Siddique, 2026, Slide 11, p.34). The lecture uses clothing layers as the analogy: each layer adds behaviour without changing the person beneath (Siddique, 2026, Slide 11, p.35-36).

**Phase 1 weakness addressed:** Phase 1 had no audit logging for report generation or export actions. UC12 explicitly requires that export actions be logged for audit purposes. Adding logging directly to ReportEngine would violate SRP — analytics is ReportEngine's responsibility, not logging. Subclassing ReportEngine into a LoggingReportEngine would lock logging to one concrete class and break if ReportEngine were replaced. The Decorator solves both problems: it wraps any IReportGenerator at runtime, adds timestamped WriteLog() calls before and after GenerateReport() and ExportReport(), and remains fully transparent to callers.

**C# implementation:** LoggingReportEngineDecorator implements IReportGenerator and holds \_inner: IReportGenerator. Every IReportGenerator method calls WriteLog() before delegating to \_inner, then calls WriteLog() again after. Program.cs wraps ReportEngine in one line: new LoggingReportEngineDecorator(new ReportEngine(15f), "audit.log"). ReportEngine is completely unchanged.

**Advantages:** satisfies UC12 audit requirement without modifying any existing class, supports decorator stacking (a TimingDecorator could wrap the LoggingDecorator), and aligns with the composition-over-inheritance principle emphasised in the lecture slides (Siddique, 2026, Slide 11, p.45).

**Disadvantage:** the decorator order matters because logging before timing produces different entries than timing before logging, deep stacking makes the call stack harder to inspect without a debugger.

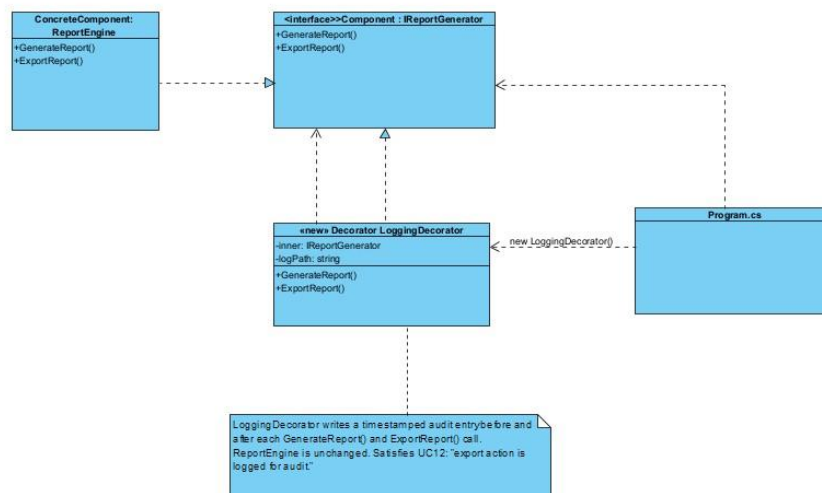


Figure 11. Pattern 5: Decorator pattern structure diagram showing LoggingReportEngineDecorator wrapping IReportGenerator, with ReportEngine as the ConcreteComponent.

## Task 7: Comparative Design Analysis (Before vs After Patterns)

This section provides a critical structural comparison of the MAS design before and after the application of GoF patterns. All comparisons reference concrete file names, line counts, and class counts from the submitted C# codebase.

### 7.1 Changes to Class Structure and Responsibility Allocation

Phase 1 delivered 5 service files totalling approximately 900 lines. The most significant structural problem was responsibility bundling: AuthService.cs contained both session management and the SeedData factory class, and ExportService.cs contained all three export format implementations as private methods. Phase 2 expanded the Services layer to 12 files by extracting each responsibility into a dedicated class. ExportService shrank from 418 lines to 86 lines, a 79% reduction this is because format logic moved to three separate strategy files. SeedData became its own file. AuthService became a pure Singleton session manager.

| Metric                       | Phase 1                | Phase 2                                              |
|------------------------------|------------------------|------------------------------------------------------|
| Service files                | 5                      | 12 (+7)                                              |
| ExportService.cs lines       | 418                    | 86 (79% reduction) = $\frac{418-86}{418} \times 100$ |
| Classes in AuthService.cs    | 2                      | 1 (SeedData extracted)                               |
| Interfaces in Services layer | 1                      | 2 (+ IExportStrategy)                                |
| Direct new() item calls      | 20 scattered           | Centralised in MTXItemFactory                        |
| AuthService instantiation    | new AuthService(users) | AuthService.GetInstance()                            |
| Audit logging                | None                   | LoggingReportEngineDecorator                         |

### 7.2 Improvements in Cohesion and Reductions in Coupling

Cohesion improved at both the class and file level. Phase 1 ExportService had low functional cohesion because CSV, PDF, and JSON methods shared a class boundary only by accident of feature area. Phase 2 CsvExportStrategy contains exclusively CSV logic; every method and field contributes to a single purpose. Similarly, Phase 1 AuthService mixed two unrelated lifecycle concerns which authentication (runtime) and seeding (startup). Separating them eliminates the risk of startup logic being accidentally invoked during authentication.

Coupling was reduced at three specific points. First, Program.cs depended on the AuthService constructor signature; changing that signature required changing Program.cs. The Singleton removes this dependency, now Program.cs now calls GetInstance() with no arguments. Second, MainDashboard was indirectly coupled to all three export format implementations through ExportService; Phase 2 couples it only to IExportStrategy. Third, SeedData.CreateCatalogue() was directly coupled to seven concrete item subclasses; MTXItemFactory encapsulates these dependencies so all callers receive only an MTXItem reference.

### 7.3 Impact on Extensibility and Maintainability

Extensibility improved most visibly in the export subsystem. Adding XML export in Phase 1 required editing `ExportService.cs`, adding a case to the switch block and a new private method — modifying a class with existing working logic. In Phase 2, adding `XmlExportStrategy` requires creating one new file that implements `IExportStrategy`. `ExportService`, `MainDashboardForm`, and all existing strategy tests are unaffected. This is the Open/Closed Principle in practice.

**The following scenarios illustrate the extensibility improvement concretely:**

- Add XML export: create `XmlExportStrategy.cs`: zero changes to `ExportService` or any form.
- Add `MountSkin` item type: add `MountSkin` class + one case in `MTXItemFactory.Create()`: all callers unchanged.
- Add performance timing to reports: create `TimingReportEngineDecorator`, `ReportEngine` unchanged.
- Enforce single session store: already enforced by Singleton construction, no callsite changes needed.
- Satisfy UC12 audit requirement: already met by `LoggingReportEngineDecorator` at startup.

### 7.4 Consequences for C# Implementation Complexity

The pattern application increased the file count from 5 to 12 service files, which is an increase in surface area. However, the complexity within each file decreased. A developer reading `CsvExportStrategy.cs` encounters only CSV logic. A developer reading `ExportService.cs` encounters only strategy delegation. The cognitive load required to understand, modify, or test any single class is lower in Phase 2 than in Phase 1, even though the total codebase is larger. The trade-off is that a new developer must understand the Strategy pattern to trace the call chain from `ExportService` through `IExportStrategy` to `CsvExportStrategy` and a one-time learning cost that is offset by reduced maintenance burden across every subsequent change.

## Task 8: Critical Reflection

### 8.1 Design Trade-offs Introduced by Design Patterns

#### 8.1.1 Strategy: Indirection vs Transparency

The Strategy pattern makes the export call chain less immediately readable. In Phase 1, all format logic was visible in one file. In Phase 2, a developer must follow the delegation from ExportService to IExportStrategy to the concrete class. This indirection is paid once during onboarding and offset by the elimination of merge conflicts on every format change. For a system where GGG may need to add XLSX export for Finance or XML export for ERP integration, the extensibility benefit clearly outweighs the navigational cost.

#### 8.1.2 Singleton: Testability vs Controlled Access

The Singleton introduces the most significant trade-off. Because `GetInstance()` returns a static instance, a test that calls `Login()` leaves `CurrentUser` populated for all subsequent tests unless `Logout()` is explicitly called. This creates test-order dependency which is a known weakness of the pattern noted in the course slides (Siddique, 2026, Slide 10, p.9). For the current WinForms desktop scope, where unit tests are not yet part of the submission, this is acceptable. For an ASP.NET migration supporting NFR04 (50 concurrent users), the Singleton would need replacement with a scoped DI registration per HTTP request.

#### 8.1.3 Adapter: Transitional Risk

`LegacyExportServiceAdapter` bridges the Phase 1 API to the Phase 2 interface. As the lecture slides warn, adapters can hide poor architectural decisions by making legacy code appear compatible when it should be retired (Siddique, 2026, Slide 11, p.31). The adapter is documented as a transitional measure in code comments, with the intent that `LegacyExportService` is removed once all call sites have migrated to the direct strategy injection approach. Without an explicit removal plan, adapters tend to persist indefinitely.

#### 8.1.4 Decorator: Stacking Order Dependency

The Decorator pattern supports stacking which is a `TimingDecorator` could wrap the `LoggingDecorator` which wraps `ReportEngine`. However, the order of stacking determines the log entry content: logging before timing records the duration inside the log entry, while timing before logging does not. This order dependency is not enforced by the type system and must be documented explicitly in `Program.cs`. The current implementation limits stacking to one decorator to avoid this complexity.

### 8.2 Limitations of the Final Design and Implementation

**Despite the Phase 2 improvements, five structural limitations remain.**

**In-memory persistence:** Both repositories store data in `List<T>` collections. All 250 seeded transactions are lost on application close. The repository interface pattern is in place, meaning SQL-backed implementations can be substituted without touching any service code, but the migration has not been implemented.

**Single-user session model:** The Singleton `AuthService` holds one `CurrentUser` at a time, directly conflicting with NFR04 (50 concurrent users). Resolution requires a session-token model where each `Login()` creates an isolated, token-identified session object.

**WinForms Windows dependency:** The UI layer cannot deploy on macOS or Linux. The service and model layers are platform-agnostic, making an ASP.NET Core or SPA migration feasible without changing business logic.

**Synchronous transaction ingestion:** NFR01 requires transaction data within 2 seconds of completion. The current synchronous pipeline would bottleneck under league-launch load. A message queue (RabbitMQ or Azure Service Bus) between the game server and MAS would resolve this.

**Unsalted SHA-256 passwords:** Passwords are hashed without a per-user salt, leaving them vulnerable to rainbow table attacks. BCrypt or PBKDF2 with random salt should replace SHA-256 in any production deployment.

### 8.3 Lessons Learned from Aligning UML Models with C# Code

Three specific lessons emerged from the Phase 2 experience.

**UML field names are binding implementation contracts.** The sequence diagrams were designed with assumed Report DTO (Data Transfer Object) field names are TransactionCount, BundlePct, TopItemName and that did not exist in the actual codebase. The real names were TotalTransactions, BundleSplit, and RevenueTrend. This mismatch produced 30 compiler errors when the new service files were first integrated. The lesson is clear: when a codebase already exists, UML models must be reverse-engineered from the code rather than designed independently. When the code does not yet exist, class diagram field names must be treated as binding contracts that implementation must follow exactly.

**Constructor changes are breaking changes that must appear in UML.** Refactoring AuthService to a Singleton removed its public constructor. Program.cs continued to call new AuthService(systemUsers) until the compiler surfaced the break. The UML class diagram showed only the Singleton's GetInstance() method, making the impact on existing call sites invisible. Future UML diagrams should explicitly mark private constructors and show dependency arrows from all callers to GetInstance() so that the downstream impact of the refactoring is visible before any code is written.

**Design patterns make intent self-documenting.** Before Phase 2, Phase 1's ExportService was described as exhibiting a Strategy-like dispatch which is informal language that left the design uncommitted. A future developer reading the switch block might add a new format directly rather than creating a new strategy class. By extracting IExportStrategy, the intent is encoded in the code structure. The presence of the interface communicates exactly how to extend the system to any developer familiar with GoF patterns, without requiring documentation or verbal explanation. This aligns with the lecture slides' description of patterns as providing a common vocabulary that allows complex design ideas to be communicated quickly (Siddique, 2026, Slide 09, p.32).

## Conclusion

Phase 2 of the Microtransaction Analytics System demonstrates how targeted application of GoF design patterns can resolve specific, documented structural weaknesses without discarding a working system. The five patterns applied are Strategy, Factory Method, Singleton, Adapter, and Decorator and each address a named Phase 1 weakness, are grounded in the ENSE706 lecture material produced by Dr Abubakar Siddique, and are visible in both the UML diagrams and the C# codebase.

The most impactful change was the Strategy pattern applied to ExportService. Reducing a 418-line class to an 86-line context class, while moving all format-specific logic into three independently testable strategy classes, demonstrates in concrete terms what the Open/Closed Principle achieves in practice. The Factory Method similarly decoupled item creation from Program.cs, and the Singleton formalised an enforcement guarantee that Phase 1 left implicit.

The critical reflection in Task 8 acknowledges that patterns introduce trade-offs. The Singleton creates test-order dependency. The Decorator introduces stacking-order risk. The Adapter risks becoming a permanent fixture rather than a transitional measure. Recognising these trade-offs is as important as applying the patterns correctly. Good software engineering requires not only knowing when to apply a pattern but also understanding the consequences of doing so and planning mitigations in advance.

The UML models produced in Phase 2 which include three sequence diagrams, one activity diagram, one state machine diagram, and the refined class diagram which collectively demonstrate that static and dynamic modelling serve different but complementary purposes. The sequence diagrams made the Strategy delegation chain visible. The state machine surfaced the Queued and Rejected states that Phase 1 had not formally implemented. The activity diagram confirmed that the session validation logic was correctly positioned in the control flow. Together they form a model of the system that would allow a new developer to understand both its structure and its behaviour without reading a single line of C#.

## References

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Grinding Gear Games. (2024). Path of Exile – Official site. <https://www.pathofexile.com>

Grinding Gear Games. (2026). Grinding Gear Games. <https://www.grindinggear.com/>

Martin, R. C. (2003). Agile Software Development, Principles, Patterns, and Practices. Prentice Hall.

New Zealand Companies Office. (2006). Grinding Gear Games Limited. <https://app.companiesoffice.govt.nz>

Siddique, A. (2026). Design Patterns [Lecture slides, ENSE706]. Auckland University of Technology.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.